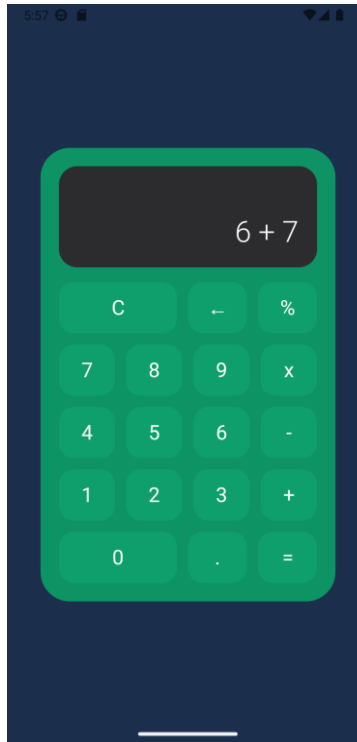




PRACTICA 02 — CALCULADORA

Al finalizar esta práctica, serás capaz de desarrollar una aplicación de calculadora utilizando **Jetpack Compose**. Toda la interfaz gráfica será creada con esta herramienta, lo que te permitirá apreciar su gran potencial y comprender cómo ha reemplazado en gran medida el desarrollo tradicional de interfaces basado en archivos *XML*. Al concluir la practica, obtendrás una aplicación similar a la que se muestra en la imagen.



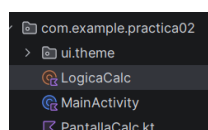
Antes de comenzar con el procedimiento, es importante mencionar que en algunas secciones únicamente se mostrará el bloque de código correspondiente. Esto se debe a que, en la práctica anterior, ya se explicó el funcionamiento y la estructura de varios de los componentes utilizados en **Jetpack Compose**, por lo que no se detallará nuevamente cada elemento paso a paso.

A lo largo de esta práctica, nos enfocaremos en explicar únicamente los conceptos y características nuevas que vayan surgiendo durante el desarrollo de la aplicación. Además, en las imágenes que incluyan fragmentos de código fuente, se incorporarán comentarios para facilitar la comprensión de las partes más importantes.

Paso 1: Iniciaremos Android Studio y seleccionamos **Empty Activity**

Paso 2: Ponemos el nombre del proyecto como **practica02**, así como la ruta donde estará mi proyecto y versión de Android la cual estaré manejando.

Paso 3: Crearemos 2 archivos en la raíz del proyecto. **PantallaCalc.kt** y **LogicaCalc.kt**.

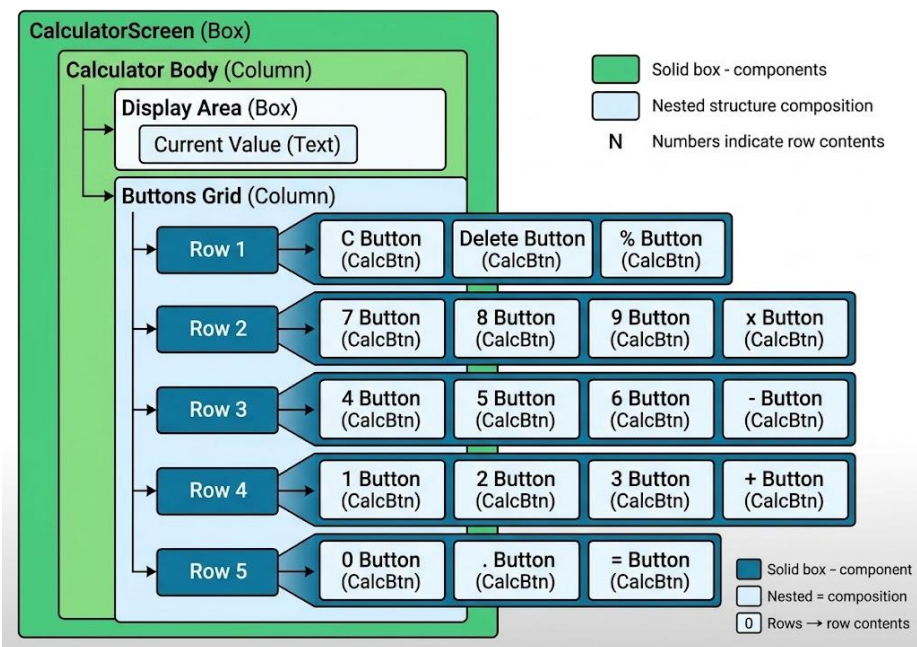




PRACTICA 02 – CALCULADORA

Como lo hemos hecho en otras practicas. El archivo **MainActivity** casi no lo tocamos, solo eliminamos las líneas que están dentro del **setContent** y todo lo que está por fuera de la clase **MainActivity**.

Paso 4: En el archivo **PantallaCalc.kt** es donde realizamos la UI de nuestra aplicación. Que se compone de la siguiente manera:



Desglose paso a paso de la estructura:

- **1. Box Principal (Fondo):** Centra la calculadora en la pantalla del dispositivo y define el color azul oscuro de fondo (*AppBackground*).
- **2. Column (Carcasa):** Representa la carcasa verde de la calculadora. Contiene un ancho fijo y esquinas redondeadas.
- **3. Box Display (Pantalla):** Se ubica en la parte superior del chasis verde con fondo oscuro (*DarkScreen*). Adentro lleva un **Text** alineado a la derecha que muestra los números ingresados.
- **4. Column Teclado:** Agrupa verticalmente las 5 filas de botones con un espacio (**spacedBy**) entre cada una.
- **5. Row (Filas de Botones):** Distribuye horizontalmente los botones dentro de cada fila. Usa *Modifier.weight()* para definir el ancho: los botones normales usan *weight(1f)* y los botones anchos ("C" y "0") usan *weight(2f)*.
- **6. CalcBtn (Componente Reutilizable):** Cada celda dentro de las filas es un **Box** individual con fondo verde claro, bordes redondeados y un **Text** al centro con su respectiva acción al hacer clic.



PRACTICA 02 — CALCULADORA



```
PantallaCalc.kt

val CalcBackground = Color(0xFF0D9364)
val DarkScreen = Color(0xFF2C2C2E)
val ButtonColor = Color(0xFF0F9F6C)
val AppBackground = Color(0xFF1B2E4B)

@Composable
fun CalculatorScreen() {
    // Declaración de variables
    var num1 by remember { mutableStateOf("") }
    var num2 by remember { mutableStateOf("") }
    var opc by remember { mutableStateOf("") }

    // Función auxiliar para procesar clicks
    fun onClick(simbolo: String) {
        when (simbolo) {
            "C" → {
                num1 = ""
                num2 = ""
                opc = ""
            }
            "←" → {
                if (num2.isNotEmpty()) num2 = num2.dropLast(1)
                else if (opc.isNotEmpty()) opc = ""
                else if (num1.isNotEmpty()) num1 = num1.dropLast(1)
            }
            "%" → {
                if (num1.isNotEmpty() && opc.isEmpty()) {
                    num1 = LogicaCalc.calcularPorcentaje(num1)
                }
            }
            "+", "-", "x", "÷" → {
                if (num1.isNotEmpty()) opc = simbolo
            }
            "=" → {
                if (num1.isNotEmpty() && num2.isNotEmpty() && opc.isNotEmpty()) {
                    num1 = LogicaCalc.calcular(num1, num2, opc)
                    num2 = ""
                    opc = ""
                }
            }
            "." → {
                if (opc.isEmpty() && !num1.contains(".")) {
                    num1 += if (num1.isEmpty()) "0." else "."
                } else if (opc.isNotEmpty() && !num2.contains(".")) {
                    num2 += if (num2.isEmpty()) "0." else "."
                }
            }
            else → { // Números (0-9)
                if (opc.isEmpty()) {
                    if (num1.length < 8) num1 += simbolo
                } else {
                    if (num2.length < 8) num2 += simbolo
                }
            }
        }
    }
}
```



PRACTICA 02 – CALCULADORA



```
PantallaCalc.kt

Box(
    modifier = Modifier
        .fillMaxSize()
        .background(AppBackground)
        .padding(16.dp),
    contentAlignment = Alignment.Center
) {
    Column(
        modifier = Modifier
            .width(320.dp)
            .clip(RoundedCornerShape(32.dp))
            .background(CalcBackground)
            .padding(20.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {
        // Pantalla
        Box(
            modifier = Modifier
                .fillMaxWidth()
                .height(110.dp)
                .clip(RoundedCornerShape(20.dp))
                .background(DarkScreen)
                .padding(20.dp),
            contentAlignment = Alignment.BottomEnd
        ) {
            Text(
                text = "$num1 $opc $num2",
                textAlign = TextAlign.End,
                color = Color.White,
                fontSize = 32.sp,
                fontWeight = FontWeight.Light,
                maxLines = 2
            )
        }
    }
}
```

PRACTICA 02 – CALCULADORA



```
PantallaCalc.kt

// Filas de los botones
Column(verticalArrangement = Arrangement.spacedBy(12.dp)) {
    // Fila 1: C, ←, %
    Row(horizontalArrangement = Arrangement.spacedBy(12.dp)) {
        CalcBtn("C", Modifier.weight(2f)) { onClick("C") }
        CalcBtn("←", Modifier.weight(1f)) { onClick("←") }
        CalcBtn("%", Modifier.weight(1f)) { onClick("%") }
    }
    // Fila 2: 7, 8, 9, x
    Row(horizontalArrangement = Arrangement.spacedBy(12.dp)) {
        CalcBtn("7", Modifier.weight(1f)) { onClick("7") }
        CalcBtn("8", Modifier.weight(1f)) { onClick("8") }
        CalcBtn("9", Modifier.weight(1f)) { onClick("9") }
        CalcBtn("x", Modifier.weight(1f)) { onClick("x") }
    }
    // Fila 3: 4, 5, 6, -
    Row(horizontalArrangement = Arrangement.spacedBy(12.dp)) {
        CalcBtn("4", Modifier.weight(1f)) { onClick("4") }
        CalcBtn("5", Modifier.weight(1f)) { onClick("5") }
        CalcBtn("6", Modifier.weight(1f)) { onClick("6") }
        CalcBtn("-", Modifier.weight(1f)) { onClick("-") }
    }
    // Fila 4: 1, 2, 3, +
    Row(horizontalArrangement = Arrangement.spacedBy(12.dp)) {
        CalcBtn("1", Modifier.weight(1f)) { onClick("1") }
        CalcBtn("2", Modifier.weight(1f)) { onClick("2") }
        CalcBtn("3", Modifier.weight(1f)) { onClick("3") }
        CalcBtn("+", Modifier.weight(1f)) { onClick("+") }
    }
    // Fila 5: 0, ., =
    Row(horizontalArrangement = Arrangement.spacedBy(12.dp)) {
        CalcBtn("0", Modifier.weight(2f)) { onClick("0") }
        CalcBtn(".", Modifier.weight(1f)) { onClick(".") }
        CalcBtn("=", Modifier.weight(1f)) { onClick("=") }
    }
}
}
```

PRACTICA 02 – CALCULADORA



```
PantallaCalc.kt

@Composable
fun CalcBtn(text: String, modifier: Modifier = Modifier, onClick: () → Unit) {
    Box(
        contentAlignment = Alignment.Center,
        modifier = modifier
            .height(56.dp)
            .clip(RoundedCornerShape(16.dp))
            .background(ButtonColor)
            .clickable { onClick() }
    ) {
        Text(text = text, fontSize = 22.sp, color = Color.White)
    }
}
```

Paso 5: En el archivo **LogicaCalc.kt** es donde realizamos lógica de nuestra aplicación. Aquí se define como un **object** en Kotlin por dos razones principales de organización y buenas prácticas:

- **Agrupación de funciones:** *LogicaCalc* contiene más de un método (**calcular** para las operaciones básicas y **calcularPorcentaje** para el porcentaje). Usar un **object** funciona como un contenedor o espacio de nombres (**namespace**) para mantener juntas todas las funciones relacionadas con la lógica matemática en un mismo lugar.
- **Patrón Singleton (instancia única):** Un **object** en Kotlin crea una instancia única que no necesita ser instanciada con `new` o `LogicaCalc()`. Permite llamar a sus métodos directamente como `LogicaCalc.calcular(...)` sin gastar memoria creando un nuevo objeto cada vez que el usuario presiona un botón.

Si la calculadora solo necesitara una única acción sencilla, una función de nivel superior (*top-level function*) bastaría. Sin embargo, al separar la lógica en un **object**, el código queda mejor estructurado, es más fácil de escalar si agregas más operaciones a futuro y facilita la lectura del proyecto.

El código sería:



PRACTICA 02 – CALCULADORA



```
LogicaCalc.kt

object LogicaCalc {

    fun calcular(num1: String, num2: String, operation: String): String {
        val n1 = num1.toDoubleOrNull() ?: return ""
        val n2 = num2.toDoubleOrNull() ?: return ""

        val result = when (operation) {
            "+" → n1 + n2
            "-" → n1 - n2
            "x" → n1 * n2
            "÷" → if (n2 ≠ 0.0) n1 / n2 else Double.NaN
            else → 0.0
        }

        return if (result % 1.0 == 0.0) {
            result.toLong().toString()
        } else {
            result.toString()
        }
    }

    fun calcularPorcentaje(num: String): String {
        val n = num.toDoubleOrNull() ?: return ""
        val result = n / 100
        return if (result % 1.0 == 0.0) result.toLong().toString() else
        result.toString()
    }
}
```

Paso 6: En el archivo **MainActivity.kt** es la parte de `setContent` solo agregaremos la función del archivo **PantallaCalc.kt**

```
MainActivity.kt

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            CalculatorScreen()
        }
    }
}
```

Paso 7: Ejecutar la aplicación y probarla.